

University of Rhode Island
Department of Electrical, Computer and Biomedical Engineering

ELE 408/409 : Embedded System Design Lecture/Laboratory

Date of Completion:

4/10/2026

Embedded AI Shopping Cart System using Edge Impulse and Raspberry Pi

Prepared by:
Joshua Torres

Prepared for: Professor Weiwei Jia
Date of Submission:

4/10/2026

Contents

1	Apparatus	3
1.1	Raspberry Pi OS	3
1.2	Pi Camera Module 3	4
1.3	Raspberry Pi 5 - 8GB	5
1.4	GeekPi MHS-3.5 inch HDMI LCD Display and Touchscreen	6
1.5	Samsung EVO Plus microSD Memory Card (128 GB)	7
2	Inventory Files and Code Analysis	8
2.1	main.py	8
2.2	inventory.py	10
2.3	cart.py	12
2.4	storage.py	14
2.5	scanner.py	16
2.6	controller.py	19
2.7	gui.py	23
2.8	tflite_infer.cpp (AI assisted)	27
2.9	System Integration - Auto Start	31
3	Ideal Improvements and Difficulties (Prototype ver.1)	33
4	Conclusion	37
5	Appendix	38

Abstract

The Embedded AI Shopping Cart system was to experiment using an AI in order to identify an item based off of a trained data set using over 300 samples utilizing Edge Impulse's image detection feature and transfer learning (learning block). Each image is classified with a label when collecting data and a background label for reference. Once the AI has recognized the image with a confidence reading of above .30, the label is then cross referenced with a JSON file which holds the inventory of the items currently being sold. If the item exists in the JSON file, then that item's name, quantity, cost and the independent running total (dependent of the quantity) will be displayed to the user using Python's Tkinter GUI package. The running total of all items are also displayed. Since this project is a prototype system at the time of writing, the user cannot "checkout" items but can void or decrement the items in the cart. The prototype also only tried to recognize one label per AI scan and not multiple or testing purposes, but later refinements may be included. The program will launch automatically once the raspberry pi's OS is fully operational.

1 Apparatus

1.1 Raspberry Pi OS

Required to operate Raspberry Pi (SBC)



Figure 1: Raspberry Pi OS

Core Software & Architecture

- **Base Distribution:** Debian 13 (Trixie)
- **Desktop Env:** PIXEL (Wayland)
- **Window Manager:** Labwc (0.8.1)
- **Kernel:** Linux 6.12 LTS
- **Arch:** 32-bit and 64-bit
- **Browsers:** Chromium, Firefox
- **UI Font:** Nunito Sans Light

System Requirements

- **RAM (Lite):** 512MB Minimum
- **RAM (Desktop):** 1 GB Min / 2 GB+
- **Storage:** 8 GB (min) / 16 GB+
- **Hardware:** Pi 1 through Pi 5

Operating Editions

- **RPi OS Lite:** Minimal CLI-only
- **RPi OS Desktop:** Default GUI
- **RPi OS Full:** Adds LibreOffice, Scratch, KiCad, Thonny, etc.

New Features & Support

- **Squeekboard:** Virtual keyboard
- **Zenoty:** Lightweight CMD dialog
- **Security:** uBlock Origin Lite
- **Boot Speed:** Taskbar optimization
- **Support Life:** Approx. 5 years

1.2 Pi Camera Module 3

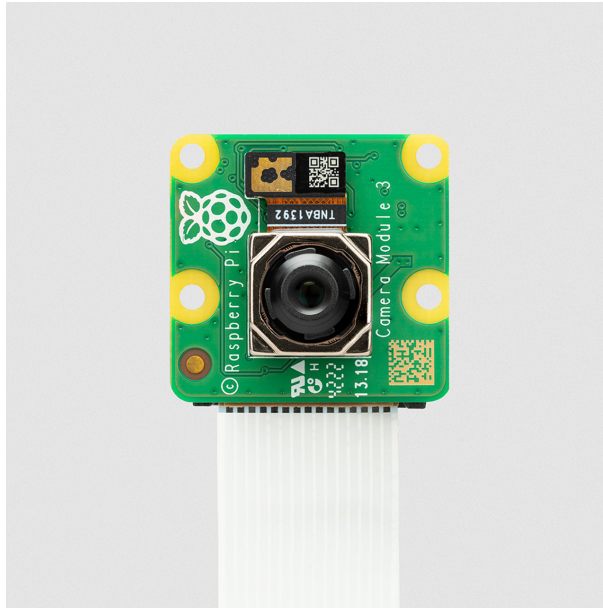


Figure 2: Pi Camera Module 3 Device

Camera Module 3 Specifications

- Focus range: 10 cm– ∞
- Focal length: 4.74 mm
- Diagonal field of view: 75°
- Horizontal field of view: 66°
- Vertical field of view: 41°
- Focal ratio (F-stop): F1.8
- Infrared-sensitive: No

Technical Specifications

- Sensor: Sony IMX708
- Resolution: 11.9 megapixels
- Sensor size: 7.4 mm diagonal
- Pixel size: $1.4 \mu\text{m} \times 1.4 \mu\text{m}$
- Horizontal/vertical: 4608×2592
- Common video modes: 1080p50, 720p100, 480p120
- Output: RAW10
- IR cut filter: Integrated
- Autofocus: Phase Detection
- Dimensions: $25 \times 24 \times 11.5$ mm
- Cable length: 200 mm
- Connector: 15×1 mm FPC

1.3 Raspberry Pi 5 - 8GB



Figure 3: Raspberry Pi 5 - 8 GB

Processor & Graphics

- **CPU:** Broadcom BCM2712 2.4 GHz
- Quad-core 64-bit Arm Cortex-A76
- 512 KB L2 per-core / 2 MB L3 cache
- **GPU:** VideoCore VII (OpenGL ES 3.1, Vulkan 1.2)
- **Video:** Dual 4Kp60 HDMI (HDR)
- **Decoder:** 4Kp60 HEVC

Memory & Connectivity

- **RAM:** 8 GB LPDDR4X-4267 SDRAM
- **Wireless:** Dual-band 802.11ac Wi-Fi
- **Bluetooth:** 5.0 / BLE
- **Ethernet:** Gigabit (PoE+ supported)
- **Storage:** microSD (SDR104 mode)

I/O & Interfaces

- **USB:** 2 × USB 3.0 (5 Gbps), 2 × USB 2.0
- **MIPI:** 2 × 4-lane Camera/Display
- **PCIe:** 2.0 x1 for peripherals
- **Header:** Raspberry Pi 40-pin GPIO

Power & Features

- **Power:** 5V/5A DC via USB-C (PD)
- **RTC:** Integrated (external battery)
- **Button:** On-board Power Button

1.4 GeekPi MHS-3.5 inch HDMI LCD Display and Touchscreen



Figure 4: GeekPi 3.5-inch Touchscreen and HDMI Display

System Features

- **OS:** Debian
- **Processor:** Broadcom (1 Count)
- **RAM:** LPDDR4
- **Connectivity:** HDMI
- **Wireless:** Bluetooth
- **USB:** 4 Total Ports
- **Compatibility:** Raspberry Pi 5

Item Details (GeekPi)

- **Model:** KZ-0060
- **Part No:** Mfr KZ-0060
- **Included Media:**
 - – ABS Case for Raspberry Pi 5
 - – 3.5 inch LCD Screen
 - – Active Cooler
 - – Stylus Pen & Screwdriver

1.5 Samsung EVO Plus microSD Memory Card (128 GB)



Figure 5: Samsung EVO Plus microSD Memory card

Performance & Capacity

- **Model:** MB-MC128SA/AM (2024)
- **Capacity:** 128 GB (64–512 GB available)
- **Read Speed:** Up to 160 MB/s
- **Write Speed:** Optimized for 4K & Apps
- **Speed Class:** U3, V30, A2, Class 10
- **Usage:** Real-time 4K UHD video

Physical Features

- **Included:** microSDXC + SD Adapter
- **Compatibility:** PCs, Consoles, Tablets

Durability (6-Proof Protection)

- **Water:** IPX7 (1m salt water, 72hrs)
- **Temp:** Operating -25°C to 85°C
- **X-Ray:** Up to 100mGy (Scanners)
- **Magnetic:** Up to 15,000 gauss
- **Drop:** Up to 5 meters (16.4 feet)
- **Wear:** Up to 10,000 swipes

Support

- **Warranty:** 10-year limited warranty

2 Inventory Files and Code Analysis

To view complete code, please see files attached to this report.

This project is a portable kiosk-style checkout prototype built around the idea of:

- using **Python** as the “brain” (GUI + business rules + inventory logic + data persistence),
- using **C++ (TensorFlow Lite)** for fast model inference on the Raspberry Pi,
- and using the **Raspberry Pi 5 + Pi Camera Module 3** as the hardware platform.

At a high level, the program runs like this:

1. The GUI opens and begins showing a live camera preview.
2. When the user presses **Scan Now (AI)**, one camera frame is captured.
3. The frame is preprocessed and passed to a compiled C++ inference runner (TensorFlow Lite).
4. The runner outputs a predicted `label` and `confidence`.
5. The Controller checks the label against the authorized inventory and updates the cart/total.
6. Manager mode allows prices/items to be modified and saved permanently to `inventory.json`.

2.1 `main.py`

`main.py` served as the entry point for the entire application. When the program was executed, this file was responsible for creating the major components of the system, connecting them together, and starting the graphical event loop. Conceptually, it functioned as the system “wiring” stage: most business rules were not implemented directly inside `main.py`, but this file instantiated the required objects (inventory, cart, scanner, controller, and GUI) and ensured that data and control flowed correctly between them.

The startup process began by configuring persistence. Because manager operations were designed to modify item prices, enable/disable items, and add or remove items, those changes needed to remain after reboot. To support this, `main.py` created a `Storage` object and called `load_items()` to attempt to load inventory data from `inventory.json`. If the JSON file existed, the stored dictionary was loaded and used to initialize the inventory state. If it did not exist, the inventory fell back to its default initial values.

After loading persisted data, `main.py` created an `Inventory` object, which acted as

the source of truth for authorized items and pricing. Next, a `Cart` object was created to represent the current order, tracking only labels and quantities. At this point, the system had both the authorized catalog (`Inventory`) and the current purchase state (`Cart`).

A scanner implementation was then selected. During early development, a `FakeScanner` was used to allow manual label entry for testing without camera hardware. In the final Raspberry Pi implementation, the scanner was set to `EdgeImpulseCppScanner`, which captured frames using `Picamera2` and performed classification by calling a compiled C++ TensorFlow Lite runner (`tf_lite_infer`). The scanner produced scan events in a consistent format as `(label, confidence)`, allowing the rest of the application to remain independent of the specific scanning method.

Once `Inventory`, `Cart`, `Storage`, and `Scanner` were created, `main.py` instantiated the `Controller`. The `Controller` enforced the application's business rules, including authorization checks, cart updates, status messages, and manager actions such as price updates. After the `Controller` was constructed, the `GUI` object was created and provided with the `Controller` reference. This design kept the `GUI` focused on display and user interaction while delegating business decisions to the `Controller`.

Finally, `main.py` started the application by calling `gui.run()`, which began Tkinter's event loop and kept the program running until the `GUI` was closed. When the program exited, the scanner's `close()` method (if implemented) was called to release the camera cleanly and prevent device-busy issues on subsequent runs.

Functions and operations used in `main.py`:

- `Storage("inventory.json")` created the persistence handler used to load and save inventory data.
- `storage.load_items()` loaded inventory contents from JSON when available.
- `Inventory(loaded_items)` constructed the authorized inventory catalog using persisted data.
- `Cart()` constructed an empty cart for the current order.
- `EdgeImpulseCppScanner(...)` created the AI scanner that used camera capture and C++ inference.
- `Controller(inventory, cart, storage, scanner)` constructed the business-logic layer connecting all core modules.
- `GUI(controller)` constructed the Tkinter interface around `Controller` actions and state.
- `gui.run()` started the Tkinter event loop and ran the application.

2.2 inventory.py

`inventory.py` represented the authorized item catalog for the shopping cart system. In practical terms, it defined the only items that were allowed to be accepted during scanning and added to the cart. This module acted as the source of truth for item information such as the display name, price, and whether an item was currently permitted for sale. By centralizing this information, the rest of the program was able to rely on Inventory whenever it needed to verify that a scanned object was valid or when it needed the correct pricing information to compute totals.

The inventory data was stored using a Python dictionary. Each entry used a unique string label as the key. This label was designed to match the classification label produced by the AI model (for example, "polar_seltzer_lime"). The associated value was a nested dictionary containing the metadata fields for that item, such as:

- **name**: the name displayed in the GUI,
- **price_cents**: the item price stored as an integer number of cents,
- **enabled**: whether the item was currently allowed to be sold,
- **ai_ready**: whether the AI model was trained/deployed to recognize that label.

Storing price as `price_cents` (instead of a floating-point dollar value) prevented rounding issues that can occur when repeatedly adding decimal currency values. In addition, separating `enabled` and `ai_ready` supported a realistic workflow: an item could exist in the inventory database but still be blocked from scanning if it was disabled by a manager or if the model had not been trained for that item yet.

Because Inventory relied on dictionary lookups as its primary operation, checking whether an item existed or retrieving its data was efficient. In average cases, dictionary access is $\mathcal{O}(1)$, meaning the time required to find an item did not scale significantly as the inventory grew. This design choice was appropriate for a shopping cart system where scan events needed to be processed quickly and repeatedly.

Functions and operations associated with `inventory.py`:

- `__init__(...)` constructed the internal dictionary of authorized items (either from defaults or loaded data).
- `is_authorized(label)` checked whether a label existed and was permitted based on the `enabled` and `ai_ready` flags.
- `get_item(label)` returned the full metadata dictionary for a label (or `None` if missing).
- `update_price(label, new_price_cents)` modified item pricing while enforcing basic validation.

-
- `add_item(...)` inserted a new item into the catalog, typically marking it as not AI-ready until training was completed.
 - `set_enabled(label, enabled)` toggled whether an item was actively sold.
 - `set_ai_ready(label, ai_ready)` controlled whether the AI system was allowed to accept that label.
 - `delete_item(label)` permanently removed an item from the catalog.

2.3 `cart.py`

`cart.py` represented the current order being built during a checkout session. In other words, it tracked what items had been accepted by the scanner/controller and how many of each item were currently in the customer's cart. This module was intentionally kept simple and focused: it did not store prices or item descriptions, because those values were already maintained in `Inventory`. Instead, the Cart only stored quantities, and totals were calculated later by combining Cart quantities with Inventory pricing.

The main data structure used in `cart.py` was a Python dictionary that mapped an item label to a quantity. For example, if `"coke_can"` was scanned twice and `"polar_seltzer_lime"` was scanned once, the cart state would be similar to:

```
{"coke_can": 2, "polar_seltzer_lime": 1}
```

This dictionary-based design allowed quantity updates and lookups to be performed efficiently (average-case $\mathcal{O}(1)$). That behavior was important because scans could occur repeatedly during a checkout session, and the system needed to update quantities quickly.

A key design detail in the cart logic was how removal was handled. Removing an item was treated differently depending on whether multiple units remained. If more than one unit existed, the quantity could be decremented immediately. However, if the quantity was exactly one, the cart logic returned a special status indicating that confirmation was needed before removing the last unit. This allowed the GUI to display an “Are you sure?” confirmation dialog, which helped prevent accidental removal of the final item of a type.

Functions and operations associated with `cart.py`:

- `__init__()` initialized an empty cart by creating an empty dictionary (`label` $\rightarrow$ `quantity`).
- `add(label: str) -> None` incremented the quantity for the given label. If the label did not exist yet, it was inserted with quantity 1.
- `get_qty(label: str) -> int` returned the current quantity of a label in the cart. If the label was not present, it returned 0. This was used frequently for quick checks (for example, blocking deletion of an item from Inventory while it was still in the cart).
- `remove_one(label: str) -> str` attempted to remove one unit of a label and returned a status string:
 - `"not_in_cart"` if the label was not present,
 - `"decremented"` if the quantity was greater than one and was decremented,
 - `"needs_confirm_last"` if the quantity was exactly one, indicating a confirmation step was needed.

-
- `remove_all(label: str) -> bool` removed the label entirely from the cart and returned `True` if removal occurred (otherwise `False` if it was not found).
 - `clear() -> None` cleared the entire cart. This supported the “Void Order” feature, which reset the current checkout session.
 - `add(label: str) -> None` incremented the quantity for the given label. If the label did not exist yet, it was inserted with quantity 1.
 - `get_qty(label: str) -> int` returned the current quantity of a label in the cart. If the label was not present, it returned 0. This was used frequently for quick checks (for example, blocking deletion of an item from Inventory while it was still in the cart).
 - `remove_one(label: str) -> str` attempted to remove one unit of a label and returned a status string:
 - `"not_in_cart"` if the label was not present,
 - `"decremented"` if the quantity was greater than one and was decremented,
 - `"needs_confirm_last"` if the quantity was exactly one, indicating a confirmation step was needed.
 - `remove_all(label: str) -> bool` removed the label entirely from the cart and returned `True` if removal occurred (otherwise `False` if it was not found).
 - `clear() -> None` cleared the entire cart. This supported the “Void Order” feature, which reset the current checkout session.

2.4 storage.py

`storage.py` provided persistence for the project by handling saving and loading of inventory data to a JSON file. Without persistence, any manager changes (such as updating prices, enabling/disabling items, or adding/removing items) would have been lost every time the Raspberry Pi rebooted or the program restarted. This file was written so that inventory data could be treated like a small database: the program could start up, load the latest saved state, and continue operating with the same configuration as before.

The storage format used was JSON (JavaScript Object Notation). JSON was appropriate for this stage of the project because it was simple, readable, and mapped directly onto Python data structures. In particular, JSON objects corresponded naturally to Python dictionaries. This made it easy to store the Inventory dictionary exactly as it existed in memory, and it made debugging straightforward because the saved file could be opened and read directly.

A design goal of `storage.py` was to keep file-handling details isolated in one place. Other modules (such as `Controller` and `Inventory`) did not need to worry about file paths, file permissions, or JSON parsing. Instead, those modules simply called `save_items(...)` or `load_items()` when needed. This kept the rest of the program cleaner and reduced the chance of bugs caused by spreading file I/O logic across multiple files.

Functions and operations associated with `storage.py`:

- `__init__(filename: str = "inventory.json")` stored the filename that would be used for persistence. The default was `inventory.json`, and the file was intended to live in the same directory where the program was executed.
- `load_items()` -> dict loaded inventory data from the JSON file:
 - * If the file did not exist, it returned an empty dictionary `{}`. This allowed the program to start cleanly on first run without crashing.
 - * If the file existed, it opened the file in read mode and parsed the JSON contents into a Python object.
 - * A basic validation step ensured the loaded data was a dictionary; if not, an empty dictionary was returned to avoid breaking the rest of the program.

-
- `save_items(items: dict) -> None` saved the current inventory dictionary to JSON:
 - * The file was opened using write mode `"w"`, meaning the file contents were overwritten each time inventory was saved. This ensured the file always represented the latest inventory state.
 - * The encoding `"utf-8"` was used so that text fields (like item names) were stored reliably using standard Unicode.
 - * `json.dump(..., indent=2, sort_keys=True)` was used so the JSON file was formatted neatly and consistently. Indentation improved readability when reviewing the file manually, and sorting keys made the file order stable across saves.

Overall, `storage.py` allowed the AI Shopping Cart to behave more like a real embedded application, where configuration changes made by a manager remained available after restart. It also established a clear path for future upgrades, since this design could later be extended to store additional information (such as order logs for refunds) or replaced with a more advanced database format (such as SQLite) without requiring major rewrites to the rest of the system.

2.5 scanner.py

`scanner.py` implemented the scanning layer for the *AI Shopping Cart* project. Its purpose was to produce scan events in a consistent format so that the rest of the program did not need to care *how* a scan was performed. Whether an item was typed manually for testing or detected using the Raspberry Pi camera and AI model, the scanner’s job was to output the same style of result: a predicted `label` and a numeric `confidence` value.

A major design goal of this file was modularity. The Controller and GUI were written so that they could accept scan events without being tied to a specific camera pipeline or machine-learning library. This was accomplished by defining a base `Scanner` interface and then providing multiple implementations. As a result, development could continue on a laptop or without camera hardware by using a fake/manual scanner, and then later the project could switch to the real AI scanner on the Raspberry Pi without rewriting the entire application.

Two scanner implementations were used:

- **FakeScanner**: used for development and debugging. It interpreted typed text as the scanned label and returned confidence as 1.0.
- **EdgeImpulseCppScanner**: used on the Raspberry Pi for AI-based scanning. It captured a frame with `Picamera2`, preprocessed it into the model’s expected format, and then called a compiled C++ TensorFlow Lite runner to perform inference.

The decision to use a C++ runner for inference was practical and based on platform constraints. `Picamera2` and `libcamera` bindings worked reliably under the Raspberry Pi’s system Python, while Python-based TensorFlow/TFLite packages were difficult to manage due to ABI compatibility issues and OS package management restrictions. Using a compiled inference runner avoided those Python dependency issues while still allowing Python to remain the overall “brain” of the shopping cart.

Functions and operations associated with `scanner.py`:

- `class Scanner`:
 - * `scan(raw_input)` defined the expected interface for any scanner implementation.
 - * It intentionally raised `NotImplementedError` so subclasses were forced to implement a real scan method.

-
- `class FakeScanner(Scanner):`
 - * `scan(raw_input)` trimmed the user-provided input string and treated it as a label.
 - * If the input was empty, it returned `(None, 0.0)` to indicate “no scan.”
 - * Otherwise, it returned `(label, 1.0)` to represent a confident manual scan.
 - `class EdgeImpulseCppScanner(Scanner):`
 - * `__init__(model_path, runner_path, warmup_sec)` initialized the camera and scanner state:
 - Started Picamera2 once at program startup (faster and avoided repeated open/close overhead).
 - Stored `last_label` and `last_conf` so the GUI could display the most recent AI result beneath the live preview.
 - Defined the temporary file location used to pass input data to the C++ runner (e.g., `/tmp/frame.int8`).
 - Loaded model quantization parameters to improve accuracy (described below).
 - * `_load_quant_params_from_runner()` retrieved the model’s true quantization parameters:
 - The C++ runner supported a special mode: `./tflite_infer --print-quant model.tflite`.
 - This printed the true input scale and zero-point used by the int8 model.
 - The scanner saved these values as `in_scale` and `in_zero`.
 - This mattered because correct quantization improved the confidence/stability of predictions. A generic mapping could still work, but it often reduced confidence.
 - * `_resize_nn(img, new_h, new_w)` performed a fast nearest-neighbor resize:
 - This avoided needing extra heavy libraries and was fast enough for real-time preview and scan behavior.
 - The model input size was fixed (96x96), so resizing was always required.

* `_preprocess_to_int8(frame)` converted a camera frame into model-ready input bytes:

- Dropped the alpha channel when the camera returned 4-channel frames.
- Converted the frame into RGB ordering.
- Resized to the model input size (96x96).
- Normalized pixel values into a float range [0, 1].
- Quantized float values into int8 using:

$$q = \text{round}\left(\frac{x}{\text{in_scale}} + \text{in_zero}\right)$$

- Clipped results into the valid int8 range $[-128, 127]$.

* `scan(...)` executed a single AI scan:

- Captured a frame using `Picamera2`.
- Preprocessed the frame into int8 bytes and wrote them to the temporary file.
- Called the C++ runner using `subprocess.run(...)`.
- Parsed the expected output format: "`<label> <confidence>`".
- Updated `last_label` and `last_conf` so the GUI could immediately show the latest prediction.
- Returned `(label, confidence)` to the Controller.

* `capture_preview_frame()` provided frames for the live camera preview:

- Returned a camera frame without running inference.
- This allowed the GUI to refresh the preview smoothly without constantly triggering scans.

* `close()` released the camera cleanly:

- This prevented “device busy” errors if the program was restarted.

Overall, `scanner.py` established a clean separation between “how an item is detected” and “what the shopping cart does after detection.” This separation supported development without hardware, supported a controlled switch to AI scanning on the Raspberry Pi, and allowed the rest of the system (Inventory, Cart, Controller, GUI) to remain stable even while the scanning approach evolved.

2.6 controller.py

`controller.py` implemented the main decision-making layer for the *AI Shopping Cart* project. This file served as the “application brain” that connected the scanning layer to the inventory rules and cart updates. The GUI was designed to remain mostly focused on display and button handling, so whenever a user pressed a button (scan, remove, void order, manager actions), the GUI forwarded that request to the Controller. The Controller then enforced the correct rules, updated the system state, and produced a clean summary of the current cart and status message for the GUI to display.

A central design idea in the Controller was that it owned the “business rules.” For example, even if the AI scanner returned a label confidently, the item still needed to be authorized in Inventory before it could be added. Similarly, manager actions needed validation and persistence so that changes would remain after reboot. By putting these rules in the Controller, the project avoided scattering logic across multiple files and kept each module’s job clear.

Functions and operations associated with `controller.py`:

- `__init__(inventory, cart, storage=None, scanner=None)`:
 - * Stored references to the core objects: `Inventory` (catalog), `Cart` (current order), optional `Storage` (JSON persistence), and optional `Scanner` (AI/manual input).
 - * Initialized `last_status`, a short message string intended for the GUI status line.
- `on_scan(label: str, confidence: float = 1.0) -> str`:
 - * Handled the “core scan decision” once a label and confidence were available.
 - * Performed the authorization check by calling `inventory.is_authorized(label)`.
 - * If unauthorized, it rejected the scan and updated `last_status` with a rejection message.
 - * If authorized, it added the item to the cart via `cart.add(label)`, and updated `last_status` with an “Added” message including the item name and price.
 - * Returned a status string (such as “accepted” or “rejected_not_authorized”) so higher layers could react if needed.
 - * (Optional improvement used later) A confidence threshold could be checked here before authorization to reject low-confidence AI results.

-
- `_save_inventory()` -> `None`:
 - * Served as a helper function to keep persistence code in one place.
 - * If a `Storage` object existed, it called `storage.save_items(inventory.authorized_Items)`.
 - * This was called after manager actions so inventory edits were written to `inventory.json`.
 - `on_remove_one(label: str) -> str`:
 - * Handled removal of one unit of an item from the cart.
 - * Called `cart.remove_one(label)`, which returned a status code:
 - `"not_in_cart"`: item not present
 - `"decremented"`: quantity > 1 so it was reduced by one
 - `"needs_confirm_last"`: quantity was exactly 1 and required confirmation
 - * Updated `last_status` depending on the outcome.
 - * Returned a controller-level status string such as `"confirm_remove_last"` so the GUI could trigger a confirmation popup when needed.
 - `confirm_remove_last(label: str, ok: bool) -> str`:
 - * Completed the “remove last item” confirmation workflow.
 - * If `ok` was false, no removal occurred and `last_status` was set to a cancellation message.
 - * If `ok` was true, it removed the label completely by calling `cart.remove_all(label)`.
 - * Returned a status string describing the result (`"removed"`, `"cancelled"`, or `"not_in_cart"`).
 - `on_void_order()` -> `None`:
 - * Voided the entire order by calling `cart.clear()`.
 - * Updated `last_status` to indicate the cart was cleared.

-
- `get_view_state()` -> dict:
 - * Returned a GUI-friendly snapshot of the current system state.
 - * Iterated through `cart.quantities` and, for each label, retrieved the matching item from Inventory using `inventory.get_item(label)`.
 - * Computed:
 - unit price from `price_cents`
 - line total = unit price $\times$ quantity
 - cart total as the sum of line totals
 - * Returned a dictionary containing:
 - `lines`: list of per-item dictionaries (label, name, qty, unit price, line total)
 - `total_cents`: total cart cost in cents
 - `status`: the `last_status` message
 - * This function allowed the GUI to avoid doing its own pricing math and instead simply render the state provided by the Controller.
 - Manager actions (inventory modifications + persistence):
 - * `manager_update_price(label, new_price_cents)` -> str:
 - Validated the new price (integer and non-negative).
 - Called `inventory.update_price(...)` to update the catalog.
 - Called `_save_inventory()` so changes persisted to JSON.
 - Updated `last_status` with a success/failure message.
 - * `manager_add_item(label, name, price_cents, enabled=False, ai_ready=False)` -> str:
 - Validated the new item fields and blocked duplicates.
 - Called `inventory.add_item(...)` to insert into the catalog.
 - Saved to JSON using `_save_inventory()`.
 - Updated `last_status` to reflect the result.
 - * `manager_set_enabled(label, enabled)` -> str:
 - Enabled or disabled an item (stop selling without deleting).
 - Called `inventory.set_enabled(...)` and saved changes.
 - * `manager_set_ai_ready(label, ai_ready)` -> str:
 - Marked an item as AI-ready or pending training.
 - This supported the real limitation that new items could not be recognized until the model was retrained.
 - Called `inventory.set_ai_ready(...)` and saved changes.

-
- * `manager_delete_item(label) -> str`:
 - Deleted an inventory item permanently.
 - Blocked deletion if the item was currently in the cart by checking `cart.get_qty(label)`.
 - Called `inventory.delete_item(...)` and saved changes.
 - Scan entry points used by the GUI:
 - * `on_scan_input(raw_input: str) -> str`:
 - Treated typed input as a manual override. If a label was typed, it forwarded directly to `on_scan(label, 1.0)`.
 - If no label was typed and a scanner existed, it optionally attempted an AI scan.
 - Updated `last_status` when no item was detected or no scanner was configured.
 - * `on_scan_now() -> str`:
 - Performed one scanner-based scan (camera/AI) by calling `scanner.scan("")`.
 - If no label was produced, it set `last_status` to indicate no item was detected.
 - If a label was produced, it forwarded the result to `on_scan(label, conf)`.

Overall, `controller.py` ensured that scanning results were not blindly trusted. Instead, each scan was filtered through authorization rules, cart update rules, and manager-controlled configuration. This design kept the program modular and made it easier to test individual components, because the Controller served as the single consistent interface between the GUI and the underlying data models.

2.7 gui.py

`gui.py` implemented the graphical interface for the *AI Shopping Cart* project using Tkinter. This file handled everything the user physically interacted with: buttons, text entry, lists, confirmation popups, and the live camera preview. The GUI was designed so that it did not contain the main “business rules” of the system. Instead, it acted as a front-end that displayed state and forwarded user actions to the Controller. This separation was important because it kept the interface code from becoming a mix of logic and display. In practice, when a button was pressed, the GUI called a Controller function, then refreshed its display using a view-state snapshot provided by the Controller.

A major feature of this GUI was the inclusion of a live camera preview. The preview was intentionally separated from inference: the preview updated continuously to show what the camera saw, but the AI model was only triggered when the user pressed **Scan Now (AI)**. This design prevented constant scanning from repeatedly adding items and also reduced unnecessary CPU usage. Under the preview, the GUI displayed the most recent predicted AI label and confidence value. These values were stored inside the scanner object as `last_label` and `last_conf`, allowing the preview to remain “live” while the scan result stayed visible between button presses.

The interface was also built to support kiosk-like behavior. The scan flow included a short cooldown (3 seconds) after an AI scan so that the same item was not accidentally scanned multiple times in rapid succession. The **End Scan** button provided a way to cancel this cooldown immediately and restore scanning if needed. Additionally, manager mode was implemented as a separate window protected by a PIN. This manager window supported inventory search, pricing updates, enabling/disabling items, AI readiness status, and adding or deleting inventory items. Because manager actions changed persistent inventory configuration, those actions were delegated to the Controller, which then saved updates to JSON through the Storage module.

Functions and operations associated with `gui.py`:

- `__init__(controller)`:
 - * Created the main Tkinter window and all widgets (labels, buttons, list-boxes, entry boxes).
 - * Stored a reference to the Controller so that GUI actions could call Controller methods.
 - * Initialized scan cooldown state variables (used for AI scanning rate-limiting).
 - * Built the camera preview panel and the “AI result” label under it.
 - * Initialized manager window state (so it could be reopened without re-entering the PIN each time).
 - * Triggered the first GUI render by calling `refresh()`.
 - * Started the preview update loop by calling `_update_preview()`.
- `run()`:
 - * Started Tkinter’s event loop using `root.mainloop()`.
 - * This call kept the program alive and responsive to user input until the window was closed.
- `refresh()`:
 - * Called `controller.get_view_state()` to retrieve a snapshot of cart contents, totals, and status.
 - * Cleared and repopulated the cart list display using the returned state.
 - * Updated the TOTAL label and the status message label.
 - * Stored a parallel list of labels (`displayed_labels`) so that clicking a row in the listbox could map back to the correct inventory label.
- `_update_preview()`:
 - * Implemented the live camera preview loop without performing inference.
 - * Called `scanner.capture_preview_frame()` to retrieve the latest frame.
 - * Converted the frame to RGB format and resized it for display.
 - * Converted the frame into a Tkinter-compatible image using Pillow (`ImageTk.PhotoImage`).
 - * Updated the preview widget and also displayed the last AI scan result using `scanner.last_label` and `scanner.last_conf`.
 - * Scheduled itself again using `root.after(...)`. This avoided blocking the GUI while still allowing a continuous preview.

-
- Manual scan input functions:
 - * `on_scan_clicked()`:
 - Read the typed label from the entry box.
 - Called `controller.on_scan_input(label)` (manual override scanning).
 - Cleared the entry box and refreshed the GUI display.
 - AI scan functions (button-based scanning):
 - * `on_scan_now_clicked()`:
 - Triggered one AI scan by calling `controller.on_scan_now()`.
 - Disabled the Scan Now button immediately to enforce a cooldown.
 - Scheduled cooldown completion using `root.after(scan_cooldown_ms, ...)`.
 - Refreshed the GUI after the scan attempt.
 - * `_end_scan_cooldown()`:
 - Re-enabled the Scan Now button once the cooldown timer finished.
 - Reset internal cooldown tracking variables so scanning could resume normally.
 - * `on_end_scan_clicked()`:
 - Cancelled any active cooldown timer (if one existed).
 - Immediately re-enabled the Scan Now button.
 - Updated the status message to indicate scanning was stopped.
 - Cart interaction functions:
 - * `on_remove_one_clicked()`:
 - Determined which cart item was selected in the listbox.
 - Called `controller.on_remove_one(label)`.
 - If the Controller requested confirmation ("`confirm_remove_last`"), a Tkinter confirmation dialog was shown.
 - If confirmed, `controller.confirm_remove_last(label, ok)` was called.
 - Refreshed the GUI afterward.
 - * `on_void_clicked()`:
 - Displayed a confirmation popup.
 - If confirmed, called `controller.on_void_order()` to clear the cart.
 - Refreshed the GUI afterward.

-
- Manager mode functions:
 - * `on_manager_clicked()`:
 - Requested a PIN using a masked input dialog.
 - If the PIN was correct, opened the manager window.
 - If the manager window was already open, brought it to the front instead of recreating it.
 - * `open_manager_window()`:
 - Constructed the manager-mode window and its widgets.
 - Populated an inventory listbox (labels) and supported search filtering.
 - Added controls for updating prices, enabling/disabling items, AI readiness, deleting items, and adding new items.
 - * Search and selection helpers:
 - `update_manager_filter()` updated the filtered inventory list based on the search text.
 - `update_manager_listbox()` re-rendered the filtered results, including enabled/AI-ready status.
 - `get_selected_manager_label()` mapped the selected list row back to its label.
 - `prefill_price_from_selection()` inserted the currently saved price into the price entry box for faster editing.
 - * Manager action handlers (each forwarded to Controller):
 - `on_manager_update_price()` called `controller.manager_update_price(...)`.
 - `on_manager_set_enabled()` called `controller.manager_set_enabled(...)`.
 - `on_manager_set_ai_ready()` called `controller.manager_set_ai_ready(...)`.
 - `on_manager_delete_item()` called `controller.manager_delete_item(...)` (with confirmation).
 - `on_manager_add_item()` called `controller.manager_add_item(...)` and then refreshed the inventory list.

Overall, `gui.py` acted as a controlled interface layer for the AI Shopping Cart system. It continuously displayed the current system state, allowed scan actions to be performed in a predictable and safe way, and supported manager configuration changes through a protected workflow. The design choice to keep scanning button-driven (instead of continuously scanning) made the shopping cart easier to demonstrate and reduced accidental repeated additions during testing.

2.8 `tflite_infer.cpp` (AI assisted)

`tflite_infer.cpp` implemented a lightweight TensorFlow Lite inference runner in C++ for the *AI Shopping Cart* project. The overall system design and workflow were defined first (camera frame → model inference → label/confidence → Controller authorization → cart update). However, the Raspberry Pi environment introduced practical issues that made a pure Python inference path difficult to maintain reliably. Because the camera stack (Picamera2/libcamera) worked consistently under system Python, but Python-based TensorFlow/TFLite tooling caused repeated installation and compatibility problems, inference was moved into a compiled C++ runner using the TensorFlow Lite library provided by the OS package manager. This approach preserved the project's intended design while improving stability on the target hardware.

This file was described as **AI assisted** because an AI tool was used specifically as a debugging and implementation aid for this C++ runner, not as the source of the project design. The design goals and the interface requirements were established ahead of time: the runner needed to accept a model file and a preprocessed input frame, execute inference, and print a simple, parseable result for the Python scanner to read. The AI tool was then used to accelerate development by helping interpret TensorFlow Lite API usage, correcting compilation/runtime issues, and responding to error messages encountered while building the runner on Raspberry Pi. In other words, the AI tool supported the implementation process by helping resolve platform-specific and library-specific problems, while the overall structure and integration approach remained consistent with the project's original design.

Why assistance was needed for this file (main difficulties): Developing `tflite_infer.cpp` involved several practical challenges that made assistance useful:

- TensorFlow Lite's C++ API required several specific setup steps (model loading, resolver selection, interpreter creation, tensor allocation). Small mistakes in this sequence could cause compile errors or runtime failures.
- Tensor shape and type expectations had to match the exported Edge Impulse model exactly (int8, [1, 96, 96, 3]). Mismatches produced confusing failures and incorrect outputs.
- Quantized models required correct handling of scale and zero-point values. Output dequantization was necessary to compute a meaningful confidence score, and input quantization consistency became important for accuracy.
- The runner needed a stable command-line interface and a predictable stdout format so that `scanner.py` could treat the program like a black-box inference engine.

Because these issues were tightly tied to library details and platform error messages, an AI tool was used as a support resource to quickly interpret the failures and propose precise code changes. This reduced time spent searching documentation and helped keep development moving while validating each change through testing on the Raspberry Pi.

How the C++ runner interacted with the Python code: The scanning pipeline in `scanner.py` was structured so that Python controlled the camera and preprocessing, then delegated inference to the C++ runner:

1. A camera frame was captured using `Picamera2`.
2. The frame was resized and formatted into the model's expected input shape ($96 \times 96 \times 3$ RGB).
3. Pixel values were normalized and quantized to `int8`.
4. The raw `int8` bytes were written to a temporary file (for example `/tmp/frame.int8`).
5. The C++ program was called using `subprocess.run(...)`.
6. The program printed one line to `stdout`: `<label> <confidence>`
7. Python parsed this line and forwarded the label/confidence to the Controller, which enforced authorization rules before updating the cart.

This interface was intentionally simple. The Python code did not need to link against ML libraries; it only needed to run a local executable and parse the result.

Program operation (normal inference mode): In inference mode the program was executed as:

```
./tflite_infer model.tflite frame.int8
```

The internal operation followed a fixed sequence:

1. Load the model from `model.tflite`.
2. Build a TensorFlow Lite interpreter and allocate tensors.
3. Validate the input tensor type (`int8`) and shape (`[1, 96, 96, 3]`).
4. Read exactly $96 \times 96 \times 3$ bytes from `frame.int8`.
5. Copy those bytes into the model input tensor.
6. Call `Invoke()` to run inference.
7. Read the output tensor (`int8`), then dequantize it using output scale and zero-point.
8. Apply softmax to convert scores into probabilities.
9. Select the maximum probability as the predicted class and confidence.

10. Print the result as: `label confidence`.

Confidence interpretation: The confidence printed by the runner represented the probability of the predicted class after softmax normalization. In practice, this value was used for two purposes:

- display feedback in the GUI (so the user could see how certain the model was),
- optional gating in the Controller to reject extremely low-confidence scans.

Quantization support and why it mattered: Because the model was quantized (int8), correct scale and zero-point values were required for meaningful results:

- Output dequantization was required to interpret the output tensor correctly and compute a valid probability distribution.
- Input quantization consistency affected classification quality. If Python quantized input pixels using generic assumptions instead of the model's true quantization parameters, confidence values and accuracy could degrade.

During testing, confidence values and prediction stability indicated that input quantization needed to be aligned more precisely with the model. To address this, a small diagnostic mode was added to the C++ runner to print the model's quantization parameters.

Additional mode added during development (quant print mode): A second command-line mode was implemented:

```
./tflite_infer --print-quant model.tflite
```

This printed the input and output quantization parameters in a predictable format:

```
INPUT <scale> <zero_point>
OUTPUT <scale> <zero_point>
```

The Python scanner used this mode once at startup to learn the true input quantization and then used those values for preprocessing. This improvement was introduced as a direct response to real debugging output and observed behavior during deployment on the Raspberry Pi, and assistance was used to quickly implement and validate the change against the TensorFlow Lite API requirements.

Functions and operations associated with `tflite_infer.cpp`:

- Model/interpreter setup:
 - * `tflite::FlatBufferModel::BuildFromFile(...)` loaded the model file.
 - * `tflite::InterpreterBuilder` constructed the interpreter using a builtin op resolver.
 - * `AllocateTensors()` allocated memory for model tensors.
- Input handling:
 - * Validated input tensor type was `int8` and shape was `[1, 96, 96, 3]`.
 - * Read raw bytes from the frame file and copied them into the input tensor buffer.
- Inference:
 - * `Invoke()` executed the model.
- Output decoding:
 - * Dequantized output using output scale and zero-point.
 - * Softmax converted logits into probabilities.
 - * The maximum probability index determined the output label and confidence.
 - * Printed `label confidence` to stdout for Python parsing.
- Quantization print mode:
 - * Printed input/output quantization parameters so Python preprocessing could match the model precisely.

Overall, `tflite_infer.cpp` provided a stable inference backend that matched the project's system design requirements. The runner's interface was intentionally minimal and predictable so that it could be called repeatedly from Python and integrated cleanly into the scanning workflow. Assistance was used primarily to respond to library-specific and platform-specific errors during implementation, and to accelerate the debugging process when dealing with TensorFlow Lite API details and quantization behavior on the Raspberry Pi.

2.9 System Integration - Auto Start

System integration for the *AI Shopping Cart* project included configuring the application to launch automatically after boot. Because the Raspberry Pi was set to desktop auto-login, the goal was for the shopping cart application to start immediately without needing to open a terminal and run `python3 main.py` manually. This step was important for making the project feel like a real embedded shopping cart system rather than a script that had to be launched by the developer each time.

Several startup methods were attempted during development. A cron `@reboot` approach was tested first, but it caused problems because cron jobs start in a limited environment that does not automatically include the graphical desktop session. In particular, GUI applications such as Tkinter require a valid display session, and cron often runs before the desktop is ready. As a result, the program either failed silently, failed to open the GUI window, or in some cases interfered with normal desktop usability. This behavior reinforced that cron is best suited for background tasks and headless scripts, not for kiosk-style GUI applications that need access to the active desktop environment.

An LXDE autostart method was also explored, but the expected directory structure was not present on the installed Raspberry Pi OS configuration. The missing session folders meant that this approach could not be relied on as a universal solution for the platform configuration being used. At that stage, startup integration needed a method that would reliably launch a graphical application after login regardless of the exact desktop session folder layout.

The final working solution used a **systemd user service**. This method was effective because a user service runs within the user's login session, meaning it had access to the correct desktop environment and could launch Tkinter normally. In addition, systemd provided a clean way to start, stop, enable, disable, and debug the application using consistent commands. This made the project easier to manage and avoided the instability observed with cron-based startup.

The systemd integration was implemented by creating a service file in the user's configuration directory:

```
/.config/systemd/user/ai_cart.service
```

The service file specified the working directory, the command used to start the application (usually a short shell script that calls `python3 main.py`), and a restart policy so that the application could automatically restart if it crashed unexpectedly.

Once the service file was created, the following systemd commands were used during setup and testing:

- Reload user services after creating or editing the service file:

```
systemctl --user daemon-reload
```

- Start the application immediately (without rebooting):

```
systemctl --user start ai_cart.service
```

- Stop the application (clean exit controlled by systemd):

```
systemctl --user stop ai_cart.service
```

- Enable auto-start so the program launched automatically at login:

```
systemctl --user enable ai_cart.service
```

- Disable auto-start so the program no longer launched automatically:

```
systemctl --user disable ai_cart.service
```

- Check whether the service was running and view its current status:

```
systemctl --user status ai_cart.service
```

A practical advantage of the systemd approach was that it supported debugging through logs. If the application failed to launch, systemd logs could be checked using `journalctl` for the user service, which provided clearer feedback than methods that failed silently. This made startup integration significantly more manageable and reduced time spent guessing why the application did not appear.

Overall, using a systemd user service provided a reliable and professional startup mechanism for the AI Shopping Cart. It ensured the application launched within the correct graphical environment after desktop auto-login, avoided the limitations of cron `@reboot` for GUI software, and provided straightforward control and debugging tools for future testing and demonstrations.

3 Ideal Improvements and Difficulties (Prototype ver.1)

There are many faults within this project due to inexperience and time management.

Ideal improvements include:

- Adding more data to the Edge Impulse AI such that accuracy of label identification is efficient.
- Using an AI training model which is able to scan multiple different labels at once (Edge Impulse or Yolo).
- Using Tkinter is good for CPU usage, however more professional applications may use PyQt.
- Modifying the GUI to fit to any specific resolution automatically
- Additional Manager functions to add and remove discounts
- Save a customer purchase to a JSON file to keep record and the ability to display previous orders and issue refunds
- Email a customer receipt once the transaction completed (optional)

Difficulties while implementing project:

Several difficulties were encountered during implementation of the AI Shopping Cart project. Most issues came from two sources: (1) getting the Raspberry Pi camera pipeline to work reliably with the AI toolchain, and (2) dealing with software/environment limitations on the Raspberry Pi 5 when attempting to run Edge Impulse deployments locally. These problems required multiple iterations, debugging steps, and ultimately an improvised design choice (Python for application control and C++ for inference) in order to produce a stable working system.

1) Edge Impulse deployment difficulties (local inference vs server-style workflow) A major challenge was learning how Edge Impulse “deployment” actually worked on-device. The project goal required the model to run locally on the Raspberry Pi, but the first attempts resembled a more cloud-connected workflow (device configuration, authentication tokens, runner configuration). While Edge Impulse does support local inference, the practical setup on the Pi involved several components (runner binaries, camera capture pipeline, configuration files) that needed to match the hardware and OS environment exactly. Misconfiguration caused confusing behavior such as the runner appearing to start correctly, but failing when attempting to capture images or interface with the camera.

2) Camera interface mismatch (V4L2/GStreamer vs libcamera/Picamera2) Edge Impulse’s Linux runner used a GStreamer pipeline that attempted to read from `/dev/video0` using V4L2 formats. On Raspberry Pi 5 with the Pi Camera Module 3, the camera stack is primarily built around **libcamera** and **Picamera2**. Even though `/dev/video0` existed, the runner’s GStreamer pipeline frequently failed with errors such as capture process failures, allocation errors, or negotiation errors. This indicated that the expected V4L2 camera behavior did not match what the libcamera pipeline was providing on the Pi 5. The camera itself was working (verified by `rpicam-hello`), but the runner’s capture pipeline was not compatible with the camera stack configuration.

3) Software environment problems (Python package management restrictions) The Raspberry Pi OS environment introduced additional friction when installing machine-learning Python packages. The OS used a package-managed Python environment (PEP 668 behavior), which prevented installing packages system-wide using `pip` without special steps. This complicated attempts to install `tf-lite-runtime` and other ML-related Python modules. In practice, it became necessary to use virtual environments and/or system packages installed through `apt`. However, mixing system camera libraries (Picamera2/libcamera) with non-system Python environments created further issues.

4) ABI and compatibility issues (pyenv/venv vs system camera bindings) During development, different Python versions and environments were tested (system Python, venv, pyenv). The camera libraries (`picamera2` and the underlying libcamera bindings) depended on compiled modules that were matched to the system’s Python build. When attempting to run camera code under a different interpreter (for example, pyenv Python), imports failed because the compiled module `libcamera._libcamera` could not be found or could not load correctly. This created a major constraint: the camera capture needed to run under system Python, while ML packages were easier to install under isolated environments. This mismatch made a pure Python “camera + inference” pipeline unreliable.

5) TensorFlow Lite runtime availability and NumPy compatibility issues Installing `tf-lite-runtime` through `pip` was not straightforward on the Pi’s OS/Python combination. In some cases no matching wheel was available for the Python version being used, and in other cases NumPy version mismatches occurred (modules compiled against NumPy 1.x failing under NumPy 2.x). These issues produced runtime import errors and prevented a clean Python-only inference solution.

6) Why improvisation was required (final working approach) Because Edge Impulse runner capture was unreliable with the Pi 5 camera stack, and Python ML environments conflicted with system camera bindings, the project required an improvised but structured solution:

- System Python + Picamera2 were used to reliably capture camera frames.
- A compiled C++ TensorFlow Lite runner (`tf_lite_infer`) was used to perform inference without depending on fragile Python ML installs.
- Python called the C++ runner using `subprocess.run(...)` and parsed a simple stdout result (`label confidence`).

This preserved the original architecture goal (local inference, no server dependency during scanning) while avoiding the platform incompatibilities encountered earlier.

7) Large-scale project difficulty (Edge Impulse model workflow and scaling concerns) Even with a working deployment, Edge Impulse introduced practical limitations when thinking about scaling beyond a small demo:

- Each new item class required collecting new training images and retraining the model.
- Training data had to cover real variation (lighting, angles, background clutter) or confidence and accuracy dropped.
- Similar-looking items (such as two flavors of the same brand) required more careful dataset design and more samples.
- The manager “add item” feature could not automatically make the AI recognize the item; it could only update the inventory database. The AI model still needed to be retrained and redeployed before the label could be reliably detected.

This made it clear that a real store-scale solution would require stronger dataset management, more classes, more training, and possibly additional sensors (weight sensors, barcodes, RFID) if full reliability was required. For the course project scope, the system was intentionally limited to a small set of items for demonstration.

Packages, tools, and system components installed/used during development:

The following tools and packages were installed or used to support development, debugging, camera testing, and inference:

- **Camera bring-up and verification:**

- * `rpikam-hello` (used to verify the camera module was detected and functioning)
- * `python3-picamera2` (Picamera2 library for capturing frames in Python)
- * `v4l2-ctl` (used to inspect `/dev/video0` formats during debugging)
- * `gstreamer` tools (`gst-launch-1.0`) (used to test camera pipelines and diagnose runner failures)

- **Python GUI and image display:**

- * `Tkinter` (used for the Shopping Cart GUI; available through system Python)
- * `python3-pil` and `python3-pil.imageTk` (Pillow libraries used to display live preview frames in Tkinter)

- **C++ inference build tools:**

- * `g++` and build tools (`build-essential`)
- * TensorFlow Lite development libraries (used to compile and link `tf_lite_infer.cpp`)
- * `pthread` linking (required by the TensorFlow Lite library)

- **Edge Impulse utilities:**

- * Edge Impulse Linux runner and CLI tools (used to attempt direct camera inference and manage deployments)
- * Device configuration file (`edge-impulse-config.json`) created during setup

- **System maintenance commands used:**

- * `sudo apt update / sudo apt install ...` for installing packages
- * `chmod +x ...` for enabling script execution
- * `systemctl --user ...` for controlling the auto-start service

Overall, the main difficulties were not caused by the high-level software logic, but by platform integration: aligning camera capture methods, deployment tooling, Python environments, and inference libraries in a way that remained stable on Raspberry Pi 5. The final architecture solved these issues by cleanly separating responsibilities between Python (application + camera) and C++ (inference).

4 Conclusion

The AI Shopping Cart project successfully demonstrated a portable kiosk-style checkout prototype on the Raspberry Pi 5 using an embedded vision workflow. The final system provided a user-friendly touchscreen GUI that displayed a live camera preview, performed button-controlled AI scans, enforced an authorized inventory list, and maintained a running cart total. Manager mode supported real configuration tasks such as updating prices, enabling or disabling items, marking items as AI-ready, adding new items, deleting items, and saving all inventory changes persistently using a JSON file.

A major outcome of the project was learning how to integrate real hardware and AI tooling in a constrained embedded environment. While Edge Impulse provided a strong training and deployment workflow, practical compatibility issues on Raspberry Pi 5 (camera capture pipelines, environment restrictions, and package conflicts) required design adaptation. The final implementation solved these constraints by using system Python with Picamera2 for reliable camera capture and a compiled C++ TensorFlow Lite runner for stable, local inference. Overall, the project achieved its intended functionality, demonstrated meaningful embedded systems integration skills, and established a clear foundation for future improvements such as expanded datasets, stronger confidence/stability gating, and additional sensing methods for higher reliability.

5 Appendix

References

- [1] Raspberry Pi Imager: <https://www.raspberrypi.com/software/>
- [2] Raspberry Pi website: <https://www.raspberrypi.com/products/camera-module-3/>
- [3] PiShop USA Store:
<https://www.pishop.us/product/raspberry-pi-camera-module-3/?srsltid=AfmB0ooXEa8y870izBPIg0058kUkKFgZQt6SFqlzm7P61zW0bcGtL2Jo>
- [4] GeekPi MHS-3.5 inch HDMI LCD Display and Touch-screen Store Page:
https://www.amazon.com/dp/B0D7VDWBBC?ref=ppx_yo2ov_dt_b_fed_asin_title
- [5] Samsung Store (128 GB microSD card):
<https://www.samsung.com/us/computing/memory-storage/memory-cards/evo-plus-adapter-microsdxc-128gb-mb-mc128sa-am.html>
- [6] Raspberry Pi News: Trixie — the new version of Raspberry Pi OS:
<https://www.raspberrypi.com/news/trixie-the-new-version-of-raspberrypi-os/>
- [7] CNX Software: Raspberry Pi OS Debian 13 "Trixie" release notes:
<https://www.cnx-software.com/2025/10/06/raspberry-pi-os-debian-13-trixie/>
- [8] Debian Project: Debian 13 "trixie" Official Release Information: <https://www.debian.org/releases/trixie/>